PROJ500 – Isaac Lab Simulation Development Scripting Guide

Plymouth Humanoid Robot Framework (PHRF)

Purpose

This document defines the **working scripting architecture used for simulation development** in the PROJ500 humanoid robotics project.

It exists so that future scripts generated by ChatGPT or other AI tools:

- Use **valid Isaac Lab APIs**
- Follow **existing project patterns**
- Avoid **deprecated Isaac Sim / Isaac Gym functions**
- Remain **compatible with current working scripts**

This guide is based on **verified working scripts currently used in the project.**

---

1. Development Environment

Simulation Framework

Component	Version / Framework
Simulator	NVIDIA Isaac Sim
RL Framework	Isaac Lab
Physics Engine	PhysX
Asset Format	USD
Robot Source	URDF → USD pipeline

Key Python Modules Used

Working scripts consistently rely on:

```
isaaclab.app
isaaclab.sim
isaaclab.assets
isaaclab.actuators
isaaclab.sim.converters
pxr (USD API)
torch
```

Important

Scripts must **launch Isaac Sim through Isaac Lab** before importing most modules.

Correct pattern:

```
from isaaclab.app import AppLauncher
```

---

2. Core Script Architecture

All working simulation scripts follow this structure.

1. CLI Argument Parsing

```
argparse
AppLauncher.add_app_launcher_args(parser)
```

Example:

```
parser = argparse.ArgumentParser()
AppLauncher.add_app_launcher_args(parser)
```

---

2. Launch Isaac Sim

This must happen **before any Isaac Lab simulation imports**.

```
app_launcher = AppLauncher(args)
simulation_app = app_launcher.app
```

Only after launching the app should the following imports occur:

```
import isaaclab.sim as sim_utils
from isaaclab.sim import SimulationContext
from isaaclab.assets import Articulation
```

---

3. Simulation Configuration

Working configuration used across scripts:

```
sim_cfg = sim_utils.SimulationCfg(
    dt=1.0 / 120.0,
    device=args.device,
    gravity=(0.0, 0.0, 0.0),
)
```

Gravity is disabled for:

- kinematic validation
- joint testing
- limit verification

---

4. Scene Setup

Typical scene components:

Ground plane

```
ground_cfg = sim_utils.GroundPlaneCfg()
ground_cfg.func("/World/ground", ground_cfg)
```

Lighting

```
light_cfg = sim_utils.DomeLightCfg(intensity=2000.0)
light_cfg.func("/World/light", light_cfg)
```

---

5. Robot Loading

Robots are loaded using the **Articulation API**.

```
robot_cfg = ArticulationCfg(
    prim_path="/World/Robot",
    spawn=sim_utils.UsdFileCfg(
        usd_path=usd_path
    )
)
```

The articulation root is already embedded in the USD.

---

6. Actuator Model

Current working actuator model:

```
ImplicitActuatorCfg
```

Example configuration:

```
ImplicitActuatorCfg(
    joint_names_expr=".*",
    effort_limit_sim=200.0,
    velocity_limit_sim=100.0,
    stiffness=500.0,
    damping=50.0,
)
```

These parameters represent **temporary generic actuator limits** used during early simulation development.

---

7. Simulation Reset

Standard reset procedure:

```
sim.reset()
robot.reset()
```

Camera positioning:

```
sim.set_camera_view([3.0,3.0,2.0],[0,0,1])
```

---

8. Commanding Joint Positions

Position commands are sent using:

```
robot.set_joint_position_target()
```

Example:

```
robot.set_joint_position_target(q_target.unsqueeze(0))
robot.write_data_to_sim()
```

9. Simulation Step Loop

Standard simulation loop:

```
while simulation_app.is_running():

    robot.set_joint_position_target(q_target.unsqueeze(0))
    robot.write_data_to_sim()

    sim.step()
    robot.update(sim_cfg.dt)
```

3. Asset Pipeline

Robot assets follow this conversion pipeline:

```
Fusion CAD
    ↓
URDF Export
    ↓
URDF Cleaning Script
    ↓
URDF → USD Conversion
    ↓
USD Fix Script
    ↓
Simulation
```

4. URDF Cleaning Script

Script:

`clean_urdf.py`

Purpose:

Removes extremely small floating-point values from URDF transforms.

Example input values:

```
-3.7e-81
0.0000000000001
```

These cause instability in simulation.

The script replaces them with zero.

Core cleaning rule:

```
if abs(value) < 1e-12 → set to 0
```

5. URDF → USD Conversion Script

Script:

`convert_urdf_to_usd.py`

Purpose:

Convert URDF robot description into a **fully functional Isaac Lab USD asset**.

Key features implemented:

- URDF conversion using Isaac Lab converter
- automatic directory creation
- sublayer path correction
- articulation root application
- coordinate system correction

Example converter configuration:

```
cfg = UrdfConverterCfg(
    asset_path=urdf_path,
    usd_dir=usd_dir,
    usd_file_name=usd_file_name,
```

```
    fix_base=False,
)
```

The script also applies default actuator drive gains.

Coordinate System Fix

Fusion 360 exports **Y-up robots**.

Isaac Sim uses **Z-up**.

The script applies a rotation:

```
Rotate X by 90°
```

This ensures the robot stands upright.

Articulation Root

The script applies the articulation root API to the robot prim.

```
UsdPhysics.ArticulationRootAPI.Apply(robot_prim)
```

6. Joint Axis Validation Script

Script:

`joint_axis_validation.py`

Purpose:

Verify that each joint rotates in the expected direction.

This script:

- disables gravity
- commands small joint movements
- cycles through positions

Sequence used:

```
neutral
+step
-step
neutral
repeat
```

Example command logic:

```
q_target[joint_i] = q_zero[joint_i] + states[state_k]
```

Where

```
states = [0, +step, -step, 0]
```

This allows visual verification of:

- joint rotation axis
- correct sign conventions
- mechanical behaviour

The robot remains floating during this test.

7. Joint Limit Validation Script

Script:

`joint_limit_validation.py`

Purpose:

Automatically test URDF joint limits in simulation.

Procedure:

For each joint:

```
neutral
lower limit
neutral
upper limit
neutral
```

Commands sent:

```
target_lower = lower_limit × limit_fraction
target_upper = upper_limit × limit_fraction
```

Default fraction:

```
0.9
```

This avoids hitting the exact limit.

Validation Logic

After commanding each target:

The script measures the reached position.

Error is calculated:

```
error = reached - target
```

Pass condition:

```
|error| ≤ tolerance
```

Default tolerance:

```
0.03 radians
```

Results Logging

Results are exported to CSV.

Example fields:

```
joint_index
joint_name
lower_limit_rad
upper_limit_rad
reached_lower_rad
reached_upper_rad
overall_pass
```

This allows verification of joint configuration accuracy.

8. Known Working APIs

The following APIs are confirmed working in the current environment.

App Launch

```
isaaclab.app.AppLauncher
```

Simulation

```
SimulationContext
SimulationCfg
PhysxCfg
```

Assets

```
Articulation
ArticulationCfg
UsdFileCfg
```

Actuators

```
ImplicitActuatorCfg
```

USD Tools

```
pxr.Usd
pxr.Sdf
pxr.UsdGeom
pxr.UsdPhysics
```

9. Coding Rules for Future Scripts

When generating new scripts for this project:

Always

✔️ Launch Isaac Sim using `AppLauncher`

✔️ Import Isaac modules **after launching the app**

✔️ Use `SimulationContext`

✔️ Use `Articulation` API for robots

✔️ Use `ImplicitActuatorCfg` unless otherwise specified

✔️ Use `robot.set_joint_position_target()` for control

✔️ Use `robot.write_data_to_sim()` before stepping

✔️ Use `robot.update(dt)` after stepping

Never

✖️ Use `omni.isaac.core` APIs

✖️ Use Isaac Gym APIs

✖️ Use `World()` class from older Isaac Sim versions

✖️ Import Isaac modules before `AppLauncher`

---

10. Purpose of This Document

This guide exists so that **future scripts generated by ChatGPT remain compatible with the working simulation architecture.**

It should be provided to AI assistants as **reference documentation** to ensure code:

- compiles correctly
- uses valid APIs
- integrates with the existing project pipeline.